

//

What is inheritance and why use it?

Inheritance means to transfer the properties of one class to another class called as inheritance. Property sender class called as parent class and property receiver class called as child class. Inheritance means acquire properties from parent class as well as add new property in child class called as inheritance.

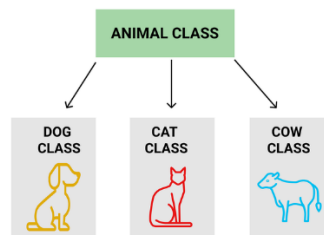
There are two benefits of inheritance:

- 1) **Reusability**: we can acquire all properties from parent class and use it called as reusability.
- 2) **Extensibility**: extensibility means we can acquire properties from parent class and add own new properties in child class called as extensibility.

The capability of a class to derive properties and characteristics from another class is called Inheritance. Inheritance is one of the most important features of Object-Oriented Programming.

- **Sub Class**: The class that inherits properties from another class is called Sub class or Derived Class.
- **Super Class**: The class whose properties are inherited by a sub-class is called Base Class or Superclass.
- **Reusability**: Inheritance supports the concept of “reusability”, i.e. when we want to create a new class and there is already a class that includes some of the code that we want, we can derive our new class from the existing class. By doing this, we are reusing the fields and methods of the existing class.

Example: Dog, Cat, Cow can be Derived Class of Animal Base Class.



Q. Why we can reuse class by using inheritance if we can reuse class by creating object?

If we reuse class by using object, then we can reuse class as it is define previously but if we reuse class by using inheritance then we can reuse its content as well as some time we can customize class content in child class as per our need as well as we can use parent content and add some own content also.

How to perform Inheritance in Java?

if we want to perform inheritance in java, we have extends keyword

```
class A{
}
class B extends A{
```

```
}
```

if we think about above code we have two classes name as A and B. Here A is parent class and B is child class.

Note: we cannot create any program in java without inheritance.

Because in java every class has a default parent class name as Object class and it is member of java.lang package and java.lang is a default package of java so not need to import it.

Explain types of inheritance in java?

Single inheritance: In single inheritance, subclasses inherit the features of one superclass.

Multilevel inheritance: In Multilevel Inheritance, a derived class will be inheriting a base class, and as well as the derived class also acts as the base class for other classes. In the below image, class A serves as a base class for the derived class B, which in turn serves as a base class for the derived class C. In Java, a class cannot directly access the grandparent's members.

Hierarchical inheritance: In Hierarchical Inheritance, one class serves as a superclass (base class) for more than one subclass. In the below image, class A serves as a base class for the derived classes B, C, and D.

Example:

```
class A {
    public void print_A() { System.out.println("Class A"); }
}
class B extends A {
    public void print_B() { System.out.println("Class B"); }
}
class C extends A {
    public void print_C() { System.out.println("Class C"); }
}
class D extends A {
    public void print_D() { System.out.println("Class D"); }
}
// Driver Class
public class Test {
    public static void main(String[] args)
    {
        B obj_B = new B();
        obj_B.print_A();
        obj_B.print_B();

        C obj_C = new C();
        obj_C.print_A();
        obj_C.print_C();

        D obj_D = new D();
        obj_D.print_A();
    }
}
```

```

        obj_D.print_D();
    }
}

```

Multiple inheritance: In Multiple inheritances, one class can have more than one superclass and inherit features from all parent classes. Please note that Java does **not** support multiple inheritances with classes. In Java, we can achieve multiple inheritances only through Interfaces. In the image below, Class C is derived from interfaces A and B.

Example:

```

// Java program to illustrate the
// concept of Multiple inheritance
import java.io.*;
import java.lang.*;
import java.util.*;

interface One {
    public void print_geek();
}
interface Two {
    public void print_for();
}

interface Three extends One, Two {
    public void print_geek();
}
class Child implements Three {
    @Override public void print_geek()
    {
        System.out.println("Geeks");
    }

    public void print_for() { System.out.println("for"); }
}

public class Main {
    public static void main(String[] args)
    {
        Child c = new Child();
        c.print_geek();
        c.print_for();
        c.print_geek();
    }
}

```

Hybrid inheritance: It is a mix of two or more of the above types of inheritance. Since Java doesn't support multiple inheritances with classes, hybrid inheritance involving multiple inheritance is also not possible with classes. In Java, we can achieve hybrid inheritance only through Interfaces if we want to involve multiple inheritance to implement Hybrid inheritance.

However, it is important to note that Hybrid inheritance does not necessarily require the use of Multiple Inheritance exclusively. It can be achieved through a combination of Multilevel Inheritance and Hierarchical Inheritance with classes, Hierarchical and Single Inheritance with classes.

Advantages Of Inheritance in Java:

1. **Code Reusability:** Inheritance allows for code reuse and reduces the amount of code that needs to be written. The subclass can reuse the properties and methods of the superclass, reducing duplication of code.
2. **Abstraction:** Inheritance allows for the creation of abstract classes that define a common interface for a group of related classes. This promotes abstraction and encapsulation, making the code easier to maintain and extend.
3. **Class Hierarchy:** Inheritance allows for the creation of a class hierarchy, which can be used to model real-world objects and their relationships.
4. **Polymorphism:** Inheritance allows for polymorphism, which is the ability of an object to take on multiple forms. Subclasses can override the methods of the superclass, which allows them to change their behaviour in different ways.

Disadvantages of Inheritance in Java:

1. **Complexity:** Inheritance can make the code more complex and harder to understand. This is especially true if the inheritance hierarchy is deep or if multiple inheritances is used.
2. **Tight Coupling:** Inheritance creates a tight coupling between the superclass and subclass, making it difficult to make changes to the superclass without affecting the subclass.

Can we create java program without inheritance explain with reason?

No, every class we write in program has a default parent class i.e. object class so in java there is already inheritance occurs.

Why Object class is parent of every class in java explain methods of object class?

Because Object class contain some inbuilt method which is required to every class in java for regular or daily uses, so Object class is parent of every class

Methods of Object class

```
int hashCode()  
String toString()  
boolean equals(Object)  
void notify()  
void notifyAll()  
void wait(int)  
void wait()  
void finalize()  
static{  
Object clone()  
Class getClass()
```

Note: in the case of inheritance we not create object of parent class we can use parent properties by using child class object.

is true if parent contain default constructor, then it is executed before child?

Yes, in Java, if the parent class contains a default constructor, it is executed before the child class's constructor. This ensures that the parent class is properly initialized before the child class's constructor is invoked.

The default constructor of the parent class is called. If the parent class also has its own parent class, then that constructor is also called, and so forth.

```
class Parent {  
    public Parent() {  
        System.out.println("Parent class constructor");  
    }  
}  
class Child extends Parent {  
    public Child() {  
        System.out.println("Child class constructor");  
    }  
}  
  
public class Main {  
    public static void main(String[] args) {  
        Child child = new Child();  
    }  
}
```

OUTPUT:

```
Parent class constructor  
Child class constructor
```

is true if parent contain parameterized constructor, then programmer must be pass parameter from child?

Yes, in Java, if the parent class contains a parameterized constructor, then when creating an instance of the child class, the programmer must explicitly call the matching constructor from the parent class. If the child class's constructor does not explicitly call the parent class's constructor, the default behaviour is to call the parent class's no-argument (default) constructor.

Using super() constructor we can pass value to parent class parameterized constructor.

when we have parameter in parent class constructor then **JVM cannot call parent constructor before child constructor in this case** child class constructor has responsibility to pass parameter to parent constructor and for that we have super() constructor.

Note: super () constructor must be first line of code in child class constructor and calling super() constructor is one type of constructor chaining using an inheritance.

Example:

```
class Parent {
    public Parent(int x) {
        System.out.println("Parent class parameterized constructor with value: " + x);
    }
}

class Child extends Parent {
    public Child(int y) {
        super(y);        // Explicitly calling the parent class's parameterized constructor
        System.out.println("Child class constructor with value: " + y);
    }
}

public class Main {
    public static void main(String[] args) {
        Child child = new Child(10);
    }
}
```

OUTPUT:

Parent class parameterized constructor with value: 10

Child class constructor with value: 10

If Child class has parameterized constructor and parent class has default constructor . will it work?

yes.

What is super() constructor and why use it?

super() constructor can call immediate parent constructor. using super constructor we can perform constructor chaining. The super keyword is used to call the constructor of its parent class to access the parent's properties and methods.

Example:

```
// superclass Person
class Person {
    Person()
    {
        System.out.println("Person class Constructor");
    }
}

class Student extends Person {
    Student()
    {
        // invoke or call parent class constructor
        super();

        System.out.println("Student class Constructor");
    }
}

// Driver Program
class Test {
    public static void main(String[] args)
    {
        Student s = new Student();
    }
}
```

Advantages of Using Java Super Keyword

The **super keyword in Java** provides many advantages in object-oriented programming are as follows:

- **Enables reuse of code:** Using the super keyword allows subclasses to inherit functionality from their parent classes, which promotes the reuse of code and reduces duplication.
- **Supports polymorphism:** Because subclasses can override methods and access fields from their parent classes using super, polymorphism is possible. This allows for more flexible and extensible code.
- **Provides access to parent class behaviour:** Subclasses can access and use methods and fields defined in their parent classes through the super keyword, which allows them to take advantage of existing behaviour without having to reimplement it.
- **Allows for customization of behaviour:** By overriding methods and using super to call the parent implementation, subclasses can customize and extend the behaviour of their parent classes.
- **Facilitates abstraction and encapsulation:** The use of super promotes encapsulation and abstraction by allowing subclasses to focus on their behaviour while relying on the parent class to handle lower-level details.

Overall, the super keyword is a key feature of inheritance and polymorphism in Java, and it provides several benefits for developers seeking to write reusable, extensible, and well-organized code.

is it possible super() constructor write on second line of code?

No, if we think about code we have get compile time error because we have super() constructor in Square class constructor but we use super() on 2nd line and the rule is it must be on first line so we break the rule so we get compile time error so if we want to solve this problem then write super() on first line.

is it possible super() and this() use at same time?

No, we cannot use this() constructor and super() constructor at same time because both constructor must be on first line of code and we cannot give single line to two different logical statement so this main reason we cannot use this() constructor and super() constructor at same time.

is it true super() pass parameter to immediate parent class?

yes. super() used to pass parameter to immediate class constructor.

what is final keyword and explain in depth?

final is non access specifier in java which we can use with variable, with function and with class i.e. final variable, final method, and final class

Final Variable:

final variable means those variables cannot modify its value once we assign it means final variable work as constant in java means if we want to fix some value then we can use final. Final variables need to initialize a value is compulsory, but static variable has not compulsion.

```
public class FinalVarApp
{
    public static void main(String x[])
    {
        final int a=100;
        ++a;
        System.out.printf("A is %d\n",a);
    }
}
```

```
C:\Program Files\Java\jdk1.8.0_291\bin>javac FinalVarApp.java
FinalVarApp.java:5: error: cannot assign a value to final variable a
        ++a;
        ^
1 error
```

if we think about above screenshot, we get compile time error because we have final variable a and we initialize value in it i.e. 100 and we have statement ++a means we try to modify the value of final

variable a and it is not possible because we cannot modify value of final variable and we try it so compiler will generate error to us.

Final method:

final method means method cannot override in child class.

Before final method we need to know what is method overriding.

Method overriding means if we define same method in parent class and redefine same method again in child class called as method overriding.

In the case of method overriding parent class method signature and child class method signature must be same or syntax must be same means **return type of method, method name, parameter type of method and parameter sequence of method must be same.**

By declaring method as final, we can avoid method overriding in java.

How can we avoid method overriding in java?

because when we declare parent class method as final then we cannot override means if we try to override parent class final method in child class then compiler will generate compile time error to us shown in following code means we can say using final method we can avoid overriding.

```
package org.techhub;
class A{
    final void show() {
        System.out.println("I am parent method show ");
    }
}
class B extends A{
    void show() {
        System.out.println("I am child method show ");
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        A b1 = new B();
        b1.show();
    }
}
```

if we think about above code, it will generate compile time error to us because we have final method show() in parent class and we try to override this method in child class B and it is not possible so compiler will generate compile time error to us.

Final class: when we declare class as final then we cannot inherit it in any another class means we can say final class has no child class means if final restrict to inheritance means final restrict to overriding because overriding cannot work without inheritance as well as final class is used for when user want to create immutable objects means immutable means we cannot modify its once we assign it.

```

package org.techhub;
final class A{
    void show() {
        System.out.println("I am parent method show ");
    }
}
class B extends A{
    void show() {
        System.out.println("I am child method show ");
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        A b1 = new B();
        b1.show();
    }
}

```

above code generate compile time error to us because we have final class A and we try to inherit it in class B and final class cannot inherit in any another class so we get compile time error

Why use method overriding and explain its benefit?

There are two benefits of method overriding

1. To customize parent logic in child class as per requirement of child.
2. To Achieve dynamic polymorphism

Method overriding means if we define same method in parent class and redefine same method again in child class called as method overriding.

In the case of method overriding parent class method signature and child class method signature must be same or syntax must be same means return type of method, method name, parameter type of method and parameter sequence of method must be same.

Suppose Consider we want to design Calculator with four operations Add, Mul, Div, Sub and we have one parent class name as Value with two methods -

void setValue(int x, int y) : here setValue() accept two parameters

int getResult(): get Result result always 0 value from Value class and we can customize getResult() method in Add class, Mul class, Div class, Sub class as per the requirement of every child class.

Source code

```

package org.techhub;
class Value{
    int a,b;
}

```

```

        void setValue(int x,int y) {
            a=x;
            b=y;
        }
        int getResult() {
            return 0;
        }
    }
    class Add extends Value{
        int getResult() {
            return a+b;
        }
    }
    class Mul extends Value{
        int getResult() {
            return a*b;
        }
    }
    public class CalcApplication {
        public static void main(String[] args) {
            Add ad = new Add();
            ad.setValue(10,20);
            int r=ad.getResult();
            System.out.printf("Addition is %d\n", r);
            Mul m = new Mul();
            m.setValue(5, 4);
            r=m.getResult();
            System.out.println("Multiplication is "+r);
        }
    }

```

if we think about above code we have parent class name as Value which contain int getResult() and this method return 0 by default and we have two child classes name as Add and Mul and we inherit Value class in Add class and override getResult() method and return a+b means change logic of getResult() in Add child class means we customize logic of Value means parent and we have one more child name as Mul where we override getResult() and return logic a*b means we change logic of parent so we can say using overriding we can customize logic of parent class.

if we want to work with method overriding, we have some important points

1. Signature of method must be same
2. If we give same name method in parent class and same name method in child class and if we change its parameter type or parameter list of parameter sequence then it is consider as overloading in inheritance.

```

package org.techhub;
class A{
    void show(int x) {
        System.out.println("I am parent method show "+x);
    }
}

```

```

    }
}
class B extends A{
    void show(float x) {
        System.out.println("I am child method show "+x);
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        B b1 = new B();
        b1.show(100);
    }
}

```

if we think about above code we have show() method in parent class and show() method in child class but we have different parameter show() in parent and show() in child so it is consider as overloading and in overloading which method should be execute is dependent on how much parameter pass in it , its sequence and its data type so if we think about main method we create here object of child class B and we call method name as b1.show(100) here parent method get executed because we have show() in parent where we provide integer as parameter.

3. If we define method as private then we cannot override it

4. If we define method in parent class as protected then we can override it as public but not vice versa.

```

package org.techhub;
class A{
    protected void show(float x) {
        System.out.println("I am parent method show "+x);
    }
}
class B extends A{
    public void show(float x) {
        System.out.println("I am child method show "+x);
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        B b1 = new B();
        b1.show(100.5f);
    }
}

```

if we think about main method we have show() method in class A which we declare as protected and we override this method as public in child class B so it is possible because public has higher priority than protected so it is possible to override protected to public but not vice versa.

Example: public to protected overriding

```

package org.techhub;
class A{

```

```

        public void show(float x) {
            System.out.println("I am parent method show "+x);
        }
    }
    class B extends A{
        protected void show(float x) {
            System.out.println("I am child method show "+x);
        }
    }
    public class OverridingApplication {
        public static void main(String[] args) {
            B b1 = new B();
            b1.show(100.5f);
        }
    }

```

if we think about above code we have method in parent class name as show() which we declare as public and we override this method as protected in child class but it is not possible so compiler generate compile time error to us

Note: we cannot override protected method as default, but we can override default method as protected.

Source code

```

package org.techhub;
class A{
    protected void show(float x) {
        System.out.println("I am parent method show "+x);
    }
}
class B extends A{
    void show(float x) {
        System.out.println("I am child method show "+x);
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        B b1 = new B();
        b1.show(100.5f);
    }
}

```

If we think about above code we have parent class name as A where we define method protected void show() and we try to override this method as default in child class but protected to default overriding is not possible so this is major reason compiler will generate compile time error to us. But we declare parent method as default and override it as protected then it is possible.

Example:

```

package org.techhub;
class A{
    void show(float x) {

```

```

        System.out.println("I am parent method show "+x);
    }
}
class B extends A{
    protected void show(float x) {
        System.out.println("I am child method show "+x);
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        B b1 = new B();
        b1.show(100.5f);
    }
}

```

Note: we can override method from default to public but not vice versa.

```

package org.techhub;
class A{
    void show(float x) {
        System.out.println("I am parent method show "+x);
    }
}
class B extends A{
    public void show(float x) {
        System.out.println("I am child method show "+x);
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        B b1 = new B();
        b1.show(100.5f);
    }
}

```

Note: we cannot override static method when we override a static then it is not consider as method overriding it is consider as method hiding.

Diving Deeper into Method Overriding

As you become more comfortable with the concept of method overriding in Java, it's time to delve into its more complex uses. Let's discuss when to use the `@Override` annotation, how method overriding works with access modifiers, and how it interacts with exception handling.

The `@Override` Annotation

The `@Override` annotation is a handy tool in Java that ensures that a method is actually overriding a method from a superclass. If it's not, the compiler will throw an error.

```

class Animal {
    void sound() {

```

```

        System.out.println("Animals make sounds");
    }
}
class Dog extends Animal {
    // @Override
    void sound() {
        System.out.println("Dogs bark");
    }
}

```

```

class HelloWorld {
    public static void main(String []args){
        // Create an instance of Dog
        Dog dog = new Dog();
        dog.sound();
    }
}

```

// Output:

// Dogs bark

In this example, the `sound()` method in the `Dog` class is annotated with `@Override`, which tells the compiler that the method should override a method in the superclass. If the `sound()` method in the `Animal` class didn't exist or had a different method signature, the compiler would throw an error.

Method Overriding and Access Modifiers

When overriding methods in Java, the access level can't be more restrictive in the subclass. For example, if the method in the parent class is `public`, the overridden method in the subclass can't be `private` or `protected`. However, it can be `public`.

Method Overriding and Exception Handling

When it comes to exception handling, the overridden method in the subclass can't throw a broader checked exception than the method in the parent class. However, it can throw narrower checked exceptions, any unchecked exceptions, or no exceptions at all.

is method overriding is beneficial every time?

While method overriding is a powerful feature in Java, it's not the only way to modify the behaviour of methods. Another related concept is method overloading, which has its own unique uses and benefits.

what is diff between static and final variable?

Final Access Modifier	Static Access Modifier
This modifier is applicable to both outer and inner classes, variables, methods, and blocks.	This modifier is only applicable to inner classes, methods, and variables.
It is not necessary to initialize the final variable at the time of its declaration.	It is necessary to initialize the static variable at the time of its declaration.
Final variable cannot be reinitialized.	Static variables can be reinitialized.
Final method cannot be inherited.	Static methods can only access the static members of the class and can only be called by other static methods.
Final class can't be inherited by any class.	The static class object can't be created and it only contains static members only.
Final keyword doesn't support any block for initialization of final variables.	Static block is used to initialize the static variables.
Final local variables are allowed.	Unlike C/C++, static local variables are not allowed in Java.
Final access modifier does not allow the use of OOP concepts like Inheritance and polymorphism.	Static access modifier allow the use of OOP concepts within static blocks of code.

Can we use static and final at same time?

yes, final keyword implies something cannot be changed. The static keyword implies class-level scope. When you combine static final in Java, you create a variable that is global to the class and impossible to change.

If the static variable declared as final, then we have to perform initialization explicitly whether we are using it or not and JVM won't provide any default value for the final static variable.

For final static variable, it is compulsory that we should perform initialization before class loading completion. We must initialize a final static variable at the time of declaration.

```
class HelloWorld {
    final static String company = "GFG";
    String name;
```



```

        int rollno;

public static void main(String[] args){
    HelloWorld ob = new HelloWorld();

    // If we create a database for GFG org then the company name should be
    //constant It can't be changed by programmer.
    // ob.company = "Geeksforgeeks"; //error: cannot assign a value to final variable company

    ob.name = "Bishal";
    ob.rollno = 007;
    System.out.println(ob.company);
    System.out.println(ob.name);
    System.out.println(ob.rollno);
}
}

```

OUTPUT:
GFG
Bishal
007

can we override final method in java?

No, The methods declared as 'final' cannot be overridden. If we declare a method as final in the parent class, then it cannot be overridden in the subclass. It is used when we don't want other classes to override the methods in the wrong way.

```

package org.techhub;
class A{
    final void show() {
        System.out.println("I am parent method show ");
    }
}
class B extends A{
    void show() {
        System.out.println("I am child method show ");
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        A b1 = new B();
        b1.show();
    }
}

```

if we think about above code it will generate compile time error to us because we have final method show() in parent class and we try to override this method in child class B and it is not possible so compiler will generate compile time error to us.

ABSTRACTION

What is abstraction in java and how we can achieve it?

abstraction means to hide implementation details from end user at designing level means in this case of abstract we do not write logic of method in abstract class just we provide its template, and we write logic of method in child class called as abstraction.

Abstract class means class cannot create its object and abstract method means method cannot have a logic or body.

Data abstraction is one of the most essential and important features of object-oriented programming in java. Data abstraction refers to providing only essential information about the data to the outside world, hiding the background details or implementation.

Consider a real-life example of a man driving a car. The man only knows that pressing the accelerator will increase the speed of the car or applying brakes will stop the car, but he does not know how on pressing the accelerator the speed is actually increasing, he does not know about the inner mechanism of the car or the implementation of an accelerator, brakes, etc. in the car. This is what abstraction is.

If we want to work with abstract class and abstract method, we have some important points.

1. abstract class cannot create its object.
2. Abstract method cannot have logics
3. If class contains abstract method, then class must be abstract means we cannot declare abstract method in non-abstract class.
4. If abstract class contains more than one abstract method then all method must be override, where abstract class get inherit and if we not required, we must be override it as blank.

Example:

```
abstract class A
{ abstract void s1();
  abstract void s2();
  abstract void s3();
}
class ADP extends A
{ void s1(){
  }
  void s2(){
  }
  void s3(){
  }
}
class B extends ADP
{
  void s1(){
    System.out.println("I required s1");
  }
}
```

```

}
class C extends ADP
{
    void s2(){
        System.out.println("I Required s2");
    }
}
public class ADPAPP
{ public static void main(String x[])
{
    B b1 = new B();
    b1.s1();
    C c1 = new C();
    c1.s2();
}
}

```

What is abstract class and abstract method?

abstract class means class cannot create its object and abstract method means method cannot have logics.

if we want to declare abstract class in java, we have abstract keyword.

Syntax:

```

abstract class classname{
    abstract returntype functionname(arguments);
}

```

Example:

```

abstract class Vehicle
{ public abstract void engine();
}
public class VehicleApp
{ public static void main(String x[])
{   Vehicle v = new Vehicle();
}
}

```

if we think about this code, we get compile time error because we try to create object of abstract class and it is not possible.

There are two benefits of abstract class and abstract methods-

- 1) To achieve abstraction**
- 2) To achieve dynamic polymorphism.**

What is diff between abstraction and encapsulation?

Abstraction	Encapsulation
Abstraction is the process or method of gaining the information.	While encapsulation is the process or method to contain the information.
In abstraction, problems are solved at the design or interface level.	While in encapsulation, problems are solved at the implementation level.
Abstraction is the method of hiding the unwanted information.	Whereas encapsulation is a method to hide the data in a single entity or unit along with a method to protect information from outside.
We can implement abstraction using abstract class and interfaces.	Whereas encapsulation can be implemented using by access modifier i.e. private, protected and public.
In abstraction, implementation complexities are hidden using abstract classes and interfaces.	While in encapsulation, the data is hidden using methods of getters and setters.
The objects that help to perform abstraction are encapsulated.	Whereas the objects that result in encapsulation need not be abstracted.
Abstraction provides access to specific part of data.	Encapsulation hides data and the user can not access same directly (data hiding).
Abstraction focus is on "what" should be done.	Encapsulation focus is on "How" it should be done.

What is diff between final and abstract method?

Final Method	Abstract Method
A final method cannot be overridden in the child class.	All the abstract methods should be overridden in the child class to provide the implementation.
A final method cannot be declared in an abstract class.	An abstract method must be declared in an abstract class.
A final method is declared using the <code>final</code> keyword.	An abstract method is declared using the <code>abstract</code> keyword.
Final methods are used to prevent the method from being overridden.	Abstract methods are used to provide a method signature without implementation.

The combination of abstract and final methods is illegal for both methods and classes. A final class cannot contain abstract methods, whereas an abstract class can contain a final method ¹.

Abstract class vs Concrete class:

An abstract class is declared using the abstract keyword and cannot be instantiated directly, while a concrete class is a type of subclass that implements all the abstract methods of its super abstract class. A concrete class can be instantiated and can also have non-abstract methods.

Abstract Class vs. Interface:

Abstract classes can have abstract and non-abstract methods, support final, non-final, static, and non-static variables, and can provide the implementation of an interface. On the other hand, interfaces can have only abstract methods (since Java 8, they can have default and static methods), support multiple inheritance, and cannot provide the implementation of an abstract class.

What is diff between abstract class and final class?

S.No.	ABSTRACT CLASS	FINAL CLASS
1.	Uses the "abstract" key word.	Uses the "final" key word.
2.	This helps to achieve abstraction.	This helps to restrict other classes from accessing its properties and methods.
3.	For later use, all the abstract methods should be overridden	Overriding concept does not arise as final class cannot be inherited
4.	A few methods can be implemented and a few cannot	All methods should have implementation
5.	Cannot create immutable objects (infact, no objects can be created)	Immutable objects can be created (eg. String class)
6.	Abstract class methods functionality can be altered in subclass	Final class methods should be used as it is by other classes
7.	Can be inherited	Cannot be inherited
8.	Cannot be instantiated	Can be instantiated
9.	Abstract class may have final methods.	Final class does not have abstract methods or final methods.
10.	Abstract class helps in to achieve Abstraction.	Final class can help to restrict the other classes from accessing the properties and methods.

Is it true abstract method must be override in child class?

yes, to write logic of that method we need to override abstract method in its child class.

Can we override constructor?

No, In Java, it is not possible to override a constructor in the same way that methods can be overridden. Constructors are special methods used to initialize objects, and they are not inherited like other methods. Each class must have its own constructor, and a subclass cannot override the constructor of its superclass.

Because constructor is same as class name defined in that class only so, It is not possible to write another name constructor in other class there is not same name of class and constructor so it is not possible to override constructor.

Can we override constructor in child class if yes then why if no then why?

No, we cannot override a constructor. Because the name of the constructor is similar to class name and overriding occurs when the same method is available in both base and derived class with different statements within.

The reason for this limitation is that constructors are not inherited by subclasses in the same way that fields and methods are. Instead, when a subclass is created, it automatically calls the constructor of its superclass. If the superclass constructor requires specific arguments, the subclass constructor must explicitly call that specific superclass constructor.

can we declare constructor as abstract explain with reason?

No, In Java, constructors cannot be declared as abstract because constructors are called to initialize an object, and abstract methods are meant to be implemented by concrete subclasses.

The constructor in an abstract class is used to initialize the variables and to achieve constructor chaining. When a subclass is instantiated, the constructor of the abstract class is invoked, and the variables are initialized. If no constructor is explicitly defined in the abstract class, the Java Virtual Machine (JVM) will provide a default constructor for the abstract class.

Why abstract classes have a constructor?

yes, we can declare constructor in abstract classes-

- The main purpose of the constructor is to initialize the newly created object. In abstract class, we have an instance variable, abstract methods, and non-abstract methods. We need to initialize the non-abstract methods and instance variables, therefore abstract classes have a constructor.
- Also, even if we don't provide any constructor the compiler will add default constructor in an abstract class.

- An abstract class can be inherited by any number of sub-classes, thus functionality of constructor present in abstract class can be used by them.
- The constructor inside the abstract class can only be called during constructor chaining i.e. when we create an instance of sub-classes. This is also one of the reasons abstract classes can have a constructor.

Why interfaces cannot have the constructor?

An Interface is a complete abstraction of class. All data members present in the interface are by default public, static, and final. All the static final fields should be assigned values at the time of declaration, otherwise it will throw compilation error saying, "variable variable_Name not initialized in default constructor".

The methods inside the interface are by default public abstract which means the method implementation cannot be provided by the interface itself, it has to be provided by the implementing class. Therefore, no need of having a constructor inside the interface.

A constructor is used to initializing non-static data members and as there are no non-static data members in the interface, there is no need of constructor.

Methods present in the interface are only declared not defined, as there is no implementation of methods, there is no need of making objects for calling methods in the interface and thus no point of having constructor in it.

If we try to create a constructor inside the interface, the compiler will give a compile-time error.

Can we declare constructor as final explain with reason?

No, In Java, constructors cannot be declared as final, primarily due to the following reasons:

1. **Inheritance and Subclassing:** The purpose of marking a method as final is to prevent it from being overridden in subclasses. However, constructors are not inherited like regular methods. Each subclass must define its own constructors to initialize its state, and these constructors are not related to the constructors of the superclass in terms of method overriding.
2. **Object Initialization:** Constructors are responsible for initializing the state of an object when it is created. If a constructor were marked as final, it would prevent subclasses from providing their own initialization logic, which goes against the concept of inheritance and polymorphism.
3. **Encapsulation Principle:** The final keyword is used to enforce the encapsulation principle by preventing certain aspects of a class from being modified or overridden. However, constructors are not meant to be overridden in the first place, so marking them as final would be redundant and unnecessary.
4. **Compiler Restrictions:** The Java language specification explicitly prohibits the use of final with constructors. Attempting to declare a constructor as final will result in a compilation error.

constructors cannot be declared as final in Java because they are not inherited like regular methods, and their purpose is to initialize the state of an object, which should not be restricted by inheritance rules. Therefore, the concept of marking constructors as final is not applicable in the context of object-oriented programming in Java.

Can we declare abstract method as static and explain with reason?

No, there are two reasons we cannot declare abstract method as static-

1) static method works with compile time polymorphism and abstract method work with dynamic polymorphism not with compile time so the behavior of static and abstract opposite of each other so we cannot declare static method as final.

2) static method allocate memory before creating object by class loader but for allocation of memory method must have definition or logic and abstract method cannot have logic or definition so we cannot declare static method as abstract

```
abstract class Vehicle{  
    static abstract void engine();  
}
```

Compiler will generate compile time error to us because we try to declare abstract method as static.

can we declare abstract method as final and explain with reason?

No, because abstract must be override in child class and final cannot override in child class so compiler will generate compile time error if we try to declare abstract method as final.

Example:

```
abstract class Vehicle{  
    final abstract void engine();  
}
```

Compiler will generate compile time error to us because we try to declare abstract method as final.

Can we declare abstract method as private and explain with reason?

No, the reason private cannot support inheritance as well as overriding and abstract cannot work without overriding and inheritance so private and abstract has opposite behavior so we cannot declare abstract method as static.


```
abstract class Vehicle  
{ private abstract void engine();  
}
```

above code will generate compile time error to us because we try to declare abstract method as private.

can we declare abstract method as protected and explain with reason?

yes, you can declare an abstract method protected. If we do so we can access it from the classes in the same package or from its subclasses. (Any you must to override an abstract method from the subclass and invoke it.)

Just to limit the access of the method to the package in which it is defined and the class which extend the abstract class as protected methods can only be accessed within same package and sub class.

Can we declare method as protected in parent and can override as public?

yes, the protected method of a superclass can be overridden by a subclass. If the superclass method is protected, the subclass overridden method can have protected or public (but not default or private) which means the subclass overridden method cannot have a weaker access specifier.

1. During **overriding**, when a variable or method is protected, it can be overridden to other subclass using either a public or protected modifier only.
2. Outer class and interface cannot be protected.

If the overridden or hidden method is public, then the overriding or hiding method must be public; otherwise, a compile-time error occurs.

If the overridden or hidden method is protected, then the overriding or hiding method must be protected or public; otherwise, a compile-time error occurs.

If the overridden or hidden method has default (package) access, then the overriding or hiding method must be protected or public, not be private; otherwise, a compile-time error occurs.

can we declare method as protected in parent and override can default?

No,

```
package org.techhub;
class A{
    protected void show(float x) {
        System.out.println("I am parent method show "+x);
    }
}
class B extends A{
    void show(float x) {
        System.out.println("I am child method show "+x);
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        B b1 = new B();
        b1.show(100.5f);
    }
}
```

If we think about above code we have parent class name as A where we define method protected void show() and we try to override this method as default in child class but protected to default overriding is not possible so this is major reason compiler will generate compile time error to us. But we declare parent method as default and override it as protected then it is possible.

can we declare method as public in parent and override as default or protected?

No, because public has higher priority than default or protected. we cannot override public method as default or protected but we can override default or protected method as public.

Can we override protected method as public and explain its reason?

Yes, the protected method of a superclass can be overridden by a subclass as public. If the superclass method is protected, the subclass overridden method can have protected or public (but not default or private) which means the subclass overridden method cannot have a weaker access specifier.

Can we override private method as protected and explain its reason?

No, you cannot override the private method in Java because it's bonded during compile time using static binding. It's also not visible on sub-class so cannot be overridden but you can hide the method by declaring another private method of the same signature.

private methods are bonded during compile time, and it is the type of the reference variable – not the type of object that it refers to – that determines what method to be called. As a side note, private methods may be performance-wise better (compared to non-private and non-final methods) due to static binding.

Can we override public method as protected or default and its' reason?

No, because public has higher priority than default or protected. we cannot override public method as default or protected but we can override default or protected method as public.

Can we declare constructor in abstract class?

Yes, we can declare constructors in an abstract class. An abstract class can have constructors, just like any other class. The constructors in an abstract class can be used to initialize the instance variables of the abstract class itself, as well as to provide a way to initialize the variables of the subclasses when they are instantiated.

```
public abstract class AbstractClass {  
    private int number;  
    public AbstractClass(int number) {  
        this.number = number;  
    }  
    public int getNumber() {  
        return number;  
    }  
    public abstract void abstractMethod();  
}
```

what is diff between overloading and overriding?

Method overloading	Method overriding
Method overloading is compile time polymorphism	Method overriding is run time polymorphism
It occurs within class or may be inheritance	Method overriding always occur using inheritance
Method overloading function name should be same but parameter or list of parameter or type or sequence must be different	In method overriding signature of parent method and signature of child method must be same
Method overloading which method should be execute is depend on parameter pass in it, its sequence and its data type	In method overriding child logics always get executed
Static binding happen using method overloading	Dynamic binding happen using method overriding
Poor perform due to the compile time polymorphism	Best perform due to run time polymorphism
Private and final method can be overload	Private and final method cannot be override
Static method can be overload	But static method cannot override means static method can hide

what is diff between static and final method?

Aspect	Static Method	Final Method
Definition	Belongs to the class, not to a specific instance.	Cannot be overridden by subclasses.
Invocation	Can be called directly on the class itself.	Called on instances of the class.
Access to Variables	Can only access static variables and methods.	Can access instance and static variables/methods.
Inheritance	Not inherited by subclasses.	Subclasses cannot override the method.
Utility	Often used for utility methods or independent tasks.	Used to enforce specific behavior in a class.

what is diff between static and abstract?

Aspect	Static Method	Abstract Method
Definition	Belongs to the class, not to a specific instance.	Declared in an abstract class and lacks a body.
Invocation	Can be called directly on the class itself.	Cannot be called directly; must be implemented by subclasses.
Access to Variables	Can only access static variables and methods.	Can access instance and static variables/methods.
Inheritance	Not inherited by subclasses.	Must be implemented by subclasses.
Body	Has a body and provides a specific implementation.	Lacks a body and requires implementation by subclasses.

What is concrete class in java?

A **concrete class** is a class that has an implementation for all of its methods. They cannot have any unimplemented methods. It can also extend an abstract class or implement an interface as long as it implements all their methods. It is a complete class and can be instantiated.

we can say that any class which is not abstract is a concrete class.

It is a complete class that can be instantiated to create objects. Some examples of concrete classes from the JDK are HashMap, HashSet, ArrayList, and LinkedList.

The code below illustrates a concrete class which extends an abstract class. The method product() in interface X is implemented by class Product, but it does not implement method sum(), therefore it must be abstract. Whereas class Main implements the unimplemented method sum(), therefore there are no unimplemented methods. Hence, it is a concrete class.

// Java program to illustrate concrete class. This is an interface-

```
interface X {  
    int product(int a, int b);  
    int sum(int a, int b);  
}
```

// This is an abstract class

abstract class can contain methods with logic? No, abstract class methods are without logic. for implementation of interface class should be abstract class? NO need of that .

```
abstract class Product implements X {  
    public int product(int a, int b) {  
        return a * b;  
    }  
}
```

// This is a concrete class that implements

```
class Main extends Product {  
    public int sum(int a, int b) {  
        return a + b;  
    }  
    public static void main(String args[]) {  
        Main ob = new Main();  
        int p = ob.product(5, 10);  
        int s = ob.sum(5, 10);  
        System.out.println("Product: " + p);  
        System.out.println("Sum: " + s);  
    }  
}
```

OUTPUT:

Product: 50

Sum: 15

What is adapter class and why use it and how to create it?

Adapter Class is a simple java class that implements an interface with **only an empty implementation**.

adapter class is intermediate class which contain all blank method of interface or abstract class, and which is able to provide specific method to its child class called as adapter class.

Why Use Adapter Classes or Pros of using Adapter classes:

- Adapter classes provide default implementations for listener interfaces, saving the need to implement all the methods of the interfaces.
- They are found in packages such as java.awt.event, java.awt.dnd, and javax.swing.event.
- Adapter classes make it easier for unrelated classes to function together and allow classes to be used in a variety of ways.
- They improve class transparency and allow the integration of related patterns within a class.
- Users have the option of using a pluggable kit to develop applications, making class usage extremely reusable.

Example:

```
abstract class A{
    abstract void s1();
    abstract void s2();
    abstract void s3();
}
class ADP extends A{
    void s1(){
    }
    void s2(){
    }
    void s3(){
    }
}
class B extends ADP{
    void s1(){
        System.out.println("I required s1");
    }
}
class C extends ADP{
    void s2(){
```

```

        System.out.println("I Required s2");
    }
}
public class ADPAPP{
    public static void main(String x[])
    {
        B b1 = new B();
        b1.s1();
        C c1 = new C();
        c1.s2();
    }
}

```

INTERFACE

What is interface in java?

Interface is same like as abstract class and it is used for achieving multiple inheritance using java as well as it is used for achieve 100% abstraction as well as it help us to achieve dynamic polymorphism. There are three reasons-

1. To Achieve 100% abstraction
2. To Achieve dynamic polymorphism
3. To achieve multiple inheritance

An interface in Java is a blueprint of a class that specifies the behaviour of a class by providing an abstract type. It is a core concept of Java's object-oriented programming and supports abstraction, polymorphism, and multiple inheritance. Interfaces are used to achieve abstraction and are implemented by classes using the implements keyword.

Interfaces in Java contain static constants and abstract methods. They cannot have method bodies, constructors, or static members. An interface is a completely "abstract class" used to group related methods with empty bodies. To access the methods defined in an interface, a class must implement the interface using the `implements` keyword.

In summary, an interface in Java is a mechanism to achieve abstraction and multiple inheritances, representing the IS-A relationship and providing a way to define the behaviour of a class.

```

// Define an interface
interface Animal {
    void animalSound(); // Abstract method
    void run(); // Abstract method
}

```

```

// Implement the interface in a class
class Dog implements Animal {
    public void animalSound() {

```

```

        System.out.println("The dog says: woof");
    }
    public void run() {
        System.out.println("The dog is running");
    }
}

```

Class	Interface
In class, you can instantiate variables and create an object.	In an interface, you can't instantiate variables and create an object.
A class can contain concrete(with implementation) methods	The interface cannot contain concrete(with implementation) methods
The access specifiers used with classes are private, protected, and public.	In Interface only one specifier is used- Public.

Is it true interface variable by default public static final?

yes, means by default interface variable are constant because they final means if we want to declare any variable within interface, we must be assign some value in it and if we not assign value to interface variable compiler will generate compile time error to us = expected.

```

2
3 interface Area{
4
5     float PI;
6 }

```

if we think about above code interface Area generate compile time error to us because PI is final variable, and we must be assign some value to it but we do not assign value so we can say interface variable must be initialize they are final.


```

1 package org.techhub;
2
3 interface Area{
4     float PI=3.14f;//public static final float PI
5 }
6 public class InterfaceApplication {
7     public static void main(String[] args) {
8
9     }
10 }

```

why?

Because we use variable within interface for constant purpose and interface cannot create its object so if we want to access variable or function without object then it must be static, so this major reason interface is variable and final.

static because we can use it by using interface name, **final** because we cannot modify its value, and **public** because it can use within package as well as outside of package.

InterfaceApplication.java

```

1 package org.techhub;
2
3 interface Area{
4     float PI=3.14f;//public static final float PI
5 }
6 public class InterfaceApplication {
7
8     public static void main(String[] args) {
9         System.out.println(Area.PI);
10    }
11 }

```

If we think about above screen short, we access Area.PI because PI is static variable so we can access it using interface name.

can define method within interface?

NO, because internally they are public abstract and abstract method cannot have logics so interface method cannot have logics.

```
interface ABC
{
    void show();
}
```

code writtern by programmer

```
Command Prompt
C:\Program Files\Java\jdk1.8.0_291\bin>javac ABC.java
C:\Program Files\Java\jdk1.8.0_291\bin>javap ABC
Compiled from "ABC.java"
interface ABC {
    public abstract void show();
}
```

code generated by compiler

If we think about above code we have above screen short we have two code 1. Written by programmer

2. Generated by compiler.

```
interface ABC{
    void show();
}
```

this code written by programmer and programmer not add public abstract with interface method and we have one code which is generated by compiler

```
interface ABC{
    public abstract void show();
}
```

so compiler by default add public abstract with void show() internally so we can say interface is internally public abstract so we can say be default interface work with abstract method so we can say it is used for achieve 100% abstraction in java.

is it true interface is used for achieving 100% abstraction?

yes, because using interface it can achieve multiple inheritance. It is also used to achieve loose coupling. Interfaces are used to implement abstraction.

```
interface Person{
    void dsplay();
}
class Student implements Person{
    public void dsplay() {
        System.out.println("This is display method of the Student class");
    }
}
class Lecturer implements Person{
    public void dsplay() {
        System.out.println("This is display method of the Lecturer class");
    }
}
public class AbstractionExample{
    public static void main(String args[]) {
        Person person1 = new Student();
        person1.dsplay();
        Person person2 = new Lecturer();
        person2.dsplay();
    }
}
```

can we declare interface method as protected?

No, we cannot declare interface method as protected and if we declare it as protected then compiler will generate compile time error.

```
interface Area{  
    protected void show();  
}
```

when we declare abstract method in abstract class then abstract method use default access specifier by default and when we declare it as protected then protected get override on default so compiler consider only once access with abstract method in abstract class so not generate compile time error.

Example:

```
abstract class A{           //internal meaning  
    abstract void show(); // default abstract void show();  
}
```

when we use protected with abstract method in abstract class then it look like as

```
abstract class A{           //internal meaning  
    protected abstract void show(); // protected abstract void show();  
}
```

Here protected keyword get override on default so compiler not generate error because compiler found only one access specifier with show () method

When we declare interface method as protected then

```
interface A{ //internal meaning  
    void show(); //public abstract void show();  
}
```

when we declare it as protected

```
interface A{  
    protected void show(); //protected public abstract void show();  
}
```

here compiler consider it as protected public so compiler found two access specifier with method at time and as per rule we cannot use two access specifier at time with function so compiler will generate compile time error to us.

can we declare interface method as static with reason?

no, because interface methods by default public abstract.

Can we declare interface method as public?

Yes, in Java, interface methods are by default public and abstract. This means that when a class implements an interface, it must provide a public implementation for all the methods declared in the interface. The public access specifier indicates that the interface methods can be accessed by classes that implement the interface, as well as any other class in the same package (or sub packages).

can we inherit one interface to another interface using extends keyword?

Yes, we can perform inheritance between interface to interface and for that we have extends keyword.

```
InterfaceTestApp.java
1 package org.techhub;
2 interface A{
3     void show();
4 }
5 interface B extends A{
6     void display(); //interface B contain two methods i.e show() and display()
7     //because B is child of A
8 }
9 class C implements B{
10     public void show() {
11         System.out.println("I am A method");
12     }
13     public void display() {
14         System.out.println("I am B method");
15     }
16 }
17 public class InterfaceTestApp {
18     public static void main(String[] args) {
19         C c1 = new C();
20         c1.show();
21         c1.display();
22     }
23 }
```

Output:

```
InterfaceTestApp.java Application
I am A method
I am B method
```

Can we implement interface in class?

In Java, you can implement an interface in a class using the `implements` keyword. When a class implements an interface, it provides concrete implementations for all the abstract methods defined in the interface. This allows the class to inherit the abstract methods from the interface and provide its own specific implementations for those methods.

```
// Define an interface
interface Animal {
    void makeSound(); // Abstract method
}
// Implement the interface in a class
class Dog implements Animal {
    public void makeSound() {
        System.out.println("Woof!");
    }
}
```

How to solve diamond problem using interface?

In the case of Java, if we think about multiple inheritance, there is more than one parent and single child, but from n number of parents, only one class is there and the remaining are interfaces. This means, suppose if we have 10 parents, then 1 must be a class and 9 should be interfaces.

So, when we have the same name method in different parents, it means different interfaces. Then the rule is we must override the interface method in its implementer class. And when we create an object of the child class or implementer class and call the overridden method, then by default the child's logic gets executed. So the compiler is not confused about method calling, so we can say using interface we resolve the diamond problem from multiple inheritance implementation.

Syntax to achieve multiple inheritances:

or

how to achieve multiple inheritance using interface?

```
class childclassname extends parentclassname implements interface1, interface.....n{

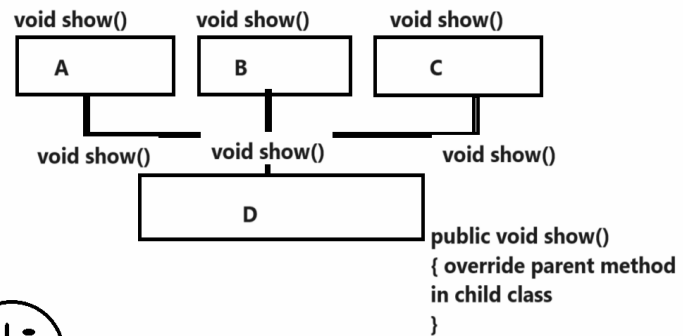
}
```

Note: when we use `extends` and `implements` at the same time, then use `extends` keyword first and after that we can use `implements` keyword.

```

1 package org.techhub;
2 interface A{
3     void show();
4 }
5 interface B{
6     void show();
7 }
8 class C{
9     void show() {
10         System.out.println("I am C method");
11     }
12 }
13 class D extends C implements A,B{
14     public void show() {
15         System.out.println("I am D method");
16     }
17 }
18 public class InterfaceTestApp {
19     public static void main(String[] args) {
20         D d1 = new D();
21         d1.show();
22     }
23 }
24

```



if we think about above screen short we have class D which is child class of C and implementer class of A and B and we override show () method in class D and when we create object of child class and try to call overridden method then by default child logics get executed so as per our example d1.show () call class D logics so there is no diamond problem occur means we can say we resolve the diamond problem using interface.

What is diff between interface and abstract class?

Interface	Abstract class
Interface use 100% abstraction	Abstract class use partial abstraction
Interface every method is by default public abstract	Need to declare manually abstract method in abstract class using abstract keyword
Interface variables are by default public static final	Abstract class variable not by default public static final
Interface is used for achieve multiple inheritance	Using abstract class we cannot achieve multiple inheritance
Interface need to implements	Abstract class need to extends or inherit
Interface cannot have constructor	Abstract class can have constructor
Interface method cannot declare as protected	Abstract class abstract method can declare as protected
Use interface keyword for interface declaration	Use class keyword for abstract class declaration
Interface contain only abstract methods ,static method and default	Abstract class can use all types of methods

Can we override static method?

No, static method we can define once and it cannot override.

Here's a summary of the reasons why static methods cannot be overridden in Java:

1. Static methods are bonded during compile time using the type of the reference variable, not the object.
2. If a subclass defines a static method with the same signature as a static method in the base class, the method in the subclass is hidden by the method in the base class.
3. Method hiding does not involve run-time polymorphism, which is a key aspect of method overriding.

What is method hiding in java and explain in detail?

Method hiding means if we override static method and if we create reference of parent class and object of child class then by default parent logics get executed means child not override its logic on parent logic means parent hide its logic from child modification called as method hiding .

```
1 package org.techhub;
2 class A{
3     static void show() {
4         System.out.println("I am parent method show ");
5     }
6 }
7 class B extends A{
8     static void show() {
9         System.out.println("I am child method show ");
10    }
11 }
12 public class OverridingApplication {
13     public static void main(String[] args) {
14         A b1 = new B();
15         b1.show();
16     }
17 }
```

<terminated> OverridingApplication [Java Application] C:\Us
I am parent method show

if we think about above code we have two classes name as class A which contain method name as static void show() and we have one more class name as B which contain method static void show() means we override static method in class B if we think about main method we create reference of class A and object of class B and we have statement b1.show() means here by default parent logics get executed means parent hide its logic from overriding by child

if we execute above code using without static method means non static method then child logics get executed means we can say non static method get override shown in following code.

```
1 package org.techhub;
2 class A{
3     void show() {
4         System.out.println("I am parent method show ");
5     }
6 }
7 class B extends A{
8     void show() {
9         System.out.println("I am child method show ");
10    }
11 }
12 public class OverridingApplication {
13     public static void main(String[] args) {
14         A b1 = new B();
15         b1.show();
16     }
17 }
--
```

<terminated> OverridingApplication [Java Applicator
I am child method show

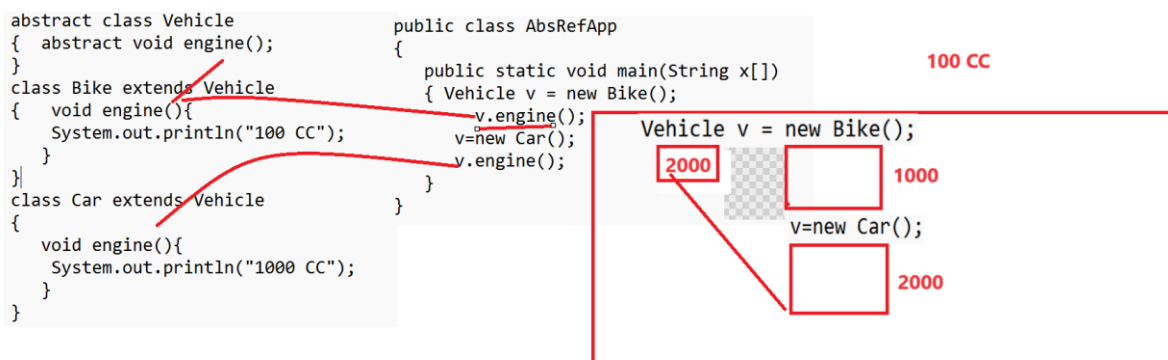
What is diff between hiding and overriding?

- In method overriding, we can use the “super” keyword to explicitly access superclass methods (super keyword is non-static reference variable we can’t use it in static method as we know static methods can only access static members of the class). Therefore, method overriding does not involve method hiding since the “super” keyword allows us to access and invoke superclass methods when needed.

Aspect	Method Overriding	Method Hiding
Type of Method	Non-static methods	Static methods
Involvement of Polymorphism	Involves run-time polymorphism	Does not involve run-time polymorphism
Determination of Method	Determined at runtime based on the actual object type	Determined at compile time based on the reference type
Inheritance Relationship	Occurs in two classes that have IS-A (inheritance) relationship	Occurs in the same class or between a class and its superclass
Example	Subclass provides a specific implementation for a method already provided by its superclass	Subclass defines a static method with the same signature as a static method in the superclass
Key Implication	Enables dynamic polymorphism	Results in method hiding and not considered method overriding

what is dynamic polymorphism and how we can achieve in java with code?

dynamic polymorphism means function bounded with object at program run time or functionality bounded with object at program run time called as dynamic polymorphism and if we want to achieve dynamic polymorphism normally we create reference of abstract class or parent class and we change object at run time and using parent reference we call method which we override in child so parent reference call method version which child class object address stored in it means parent reference decide which child logics should be execute at program run time called as dynamic polymorphism means we have same name method in different child with different logics and we call it at run time it is called as dynamic polymorphism.



if we think about above code we have abstract class name as Vehicle and we have abstract method name as abstract void engine() and we have two child classes name as Bike and Car and we inherit Vehicle class in Bike and Car and we override engine() method in Bike with logic 100 CC and engine() method in Car with logic 1000 CC and we if we think about main method we have statement `Vehicle v=new Bike()` means here we create object of Bike class and store its address i.e 1000 in our example in reference variable v means our reference work with bike object at run time and we have statement `v.engine()` so here we get output 100 CC because v contain address of Bike class and again we have statement `v= new Car()` means here we create object of Car at run time and store address Car object in reference variable v i.e 2000 means our vehicle points to Car object at run time so when we call statement `v.engine()` so v call logic from Car class means here if we think we have engine() method with two different logics means our engine() method behave differently two times here it is polymorphism because same thing behave differently but which method should call is dependent on run time as per the object of child class so we can say above example is implementation of dynamic polymorphism.

What is coupling in java and explain detail?

Here coupling means if particular method or constructor is dependent on some specified parameter called as coupling or dependency injection also.

```
class Employee
```

```
{
```

```
}
```

```
class Company
```

```
{
```

```
void setEmployee(Employee emp)
```

```
{
```

```
}
```

```
}
```

```
Company c = new Company();
```

```
Employee e = new Employee();
```

```
c.setEmployee(e)
```

example of coupling or
dependency injection

Note: if we think about setEmployee() method of Company class then we cannot call setEmployee() method without employee reference because this method is dependent on Employee class reference because Employee is parameter of setEmployee() method so it is called as coupling or dependency injection

There are two types of coupling

1. Tight Coupling: tight coupling means if particular method is dependent on specified parameter or specified object 100% called as tight coupling

Example: Suppose consider we want to design Calculator with four different operations add, mul and we design separate class for every operation means for add operation we have Add class, for Mul operation we have Mul class etc and we have one common class name as Calc Where we can perform all calculator operations or get results for different operation shown in following diagram.

```
class Add
```

```
{ int a,b;
```

```
void setValue(int x,int y)
```

```
{ a=x;
```

```
b=y;
```

```
}
```

```
int getAdd(){
```

```
return a+b;
```

```
}
```

```
}
```

```
class Mul
```

```
{ int a,b;
```

```
void setValue(int x,int y)
```

```
{ a=x;
```

```
b=y;
```

```
}
```

```
int getMul(){
```

```
return a*b;
```

```
}
```

```
}
```

```
class Calc
```

```
{ void performCalc(Add ad)
```

```
{ int result = ad.getAdd();
```

```
System.out.println("Result is "+result);
```

```
}
```

```
public class TightCouplingApp
```

```
{ public static void main(String x[])
```

```
{
```

```
Calc c = new Calc();
```

```
Add ad = new Add();
```

```
ad.setValue(10,20);
```

```
c.performCalc(ad);
```

```
Mul m = new Mul();
```

```
m.setValue(5,4);
```

```
c.performCalc(m);
```

```
}
```

```
}
```

Note: if we think about Calc class it contain one method name as performCalc(Add ad) and this method contain one parameter name as Add means this method cannot accept any other class object except Add class and if we think about main method in main method we create object of Mul class and try to pass Mul class object in performCalc() method but this method not accept Mul class object only Accept Add class object means performCalc() method 100% dependent on Add class object so it is called as tight coupling.

Generate here compile time error

If we want to solve this problem we have two approaches

a) Use function overloading i.e compile time polymorphism

If we want to solve above problem using a function overloading we have to design or define one more function name as performCalc() and pass Mul class as parameter in performCalc () function means we have two functions with same name and different parameters i.e one function use Add as parameter and one function use Mul as parameter so the benefit is we can pass any object to performCalc() shown in following code.

```
package org.techhub;
```

```
class Add{
```

```

        int a,b;
        void setValue(int x,int y) {
            a=x;
            b=y;
        }
        int getAdd() {
            return a+b;
        }
    }
    class Mul{
        int a,b;
        void setValue(int x,int y) {
            a=x;
            b=y;
        }
        int getMul() {
            return a*b;
        }
    }
    class Calc
    { void performCalc(Add ad) {
        int result=ad.getAdd();
        System.out.println("Addition is "+result);
    }
    void performCalc(Mul m) {
        int result=m.getMul();
        System.out.println("Multiplication is "+result);
    }
    }
    public class TightCouplingApplication {

        public static void main(String[] args) {
            Calc c = new Calc();
            Add ad = new Add();
            ad.setValue(10,20);
            c.performCalc(ad);
            Mul m = new Mul();
            m.setValue(5, 4);
            c.performCalc(m);
        }
    }

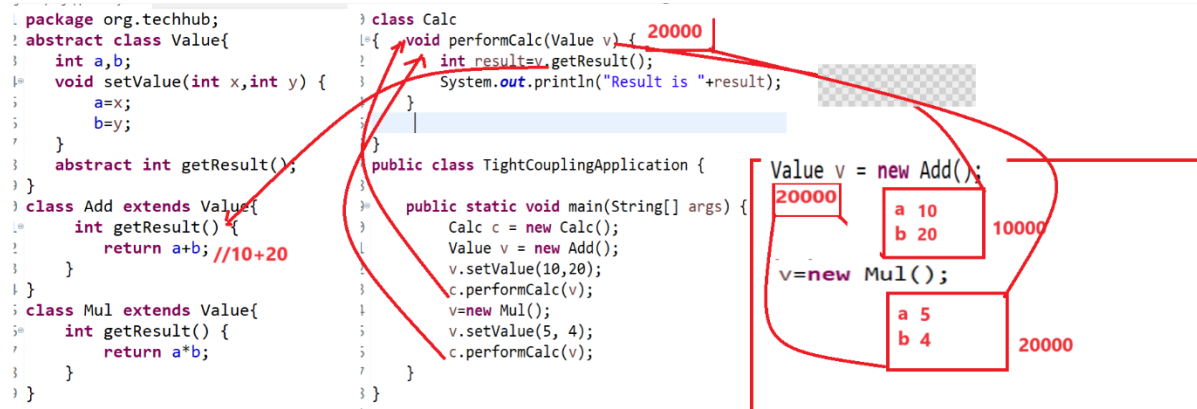
```

if we think about above code solve problem of previous program by using function overloading technique the drawback is if we have 100 operation in your Calc then we required to write 100 classes and we need to overload 100 different methods in Calc class so it is not possible in real time scenario so we want to design single method which accept the different class objects at run time means as per our example we want to design single method name as performCalc() which can accept Add class object as well as Mul class object so you can use abstraction with dynamic polymorphism and loose coupling.

shown in following diagram

b) Use Dynamic polymorphism using loose coupling technique

2. Loose Coupling: loose coupling means if particular method partially dependent on specified parameter called as loose coupling.



if we think about above diagram we have abstract class name as Value with two methods one is void setValue(int x,int y) which is non abstract method and which contain parameter and we have one method name as abstract int getResult() and this is abstract method and we inherit Value class in Add class and Value class in Mul class and we override int getResult() method in Add class and in Mul class and we write addition logic in getResult() in Add class and multiplication logic in getResult() in Mul class by using overriding and we have one more class name as Calc which contain one method name as performCalc(Value v) here we pass reference of parent class i.e Value and when we pass reference of parent then we can pass object of any child class means this method can accept all objects of Value child classes i.e Add and Mul as per our example and if we think about main method we have statement Calc c = new Calc() means we create here object of Calc class and we have statement Value v= new Add() means here we have Add class object and we store address of Add class object in value reference which is 10000 in our example means v points to 10000 address which is Add class object means v points to Add class object and we have statement c.performCalc(v) means we pass address of Add class to performCalc() function and in performCalc() we get logic v.getResult() means this statement call logic from Add class so we get Addition is 30 and again we have statement in main method name as v=new Mul() means we create new object of Mul class whose address is 20000 and we store this 20000 address in reference variable v means v release the address of Add class and points to new object means v change object at run time so we have c.performCalc(v) means we pass address of Mul class object of performCalc() method and in performCalc() we have logic v.getResult() so v call the logic from Mul class means performCalc() method work with two different object at run time and call two different version of getResult() so it is dynamic polymorphism as well as performCalc() method is not dependent on specified class object it can work with multiple object at run time as per our example it work with two objects at run time those are child of Value class so we can say performCalc() method achieve loose coupling also means we can say dynamic polymorphism can achieve by using loose coupling technique and using this approach we solve the problem of compile time polymorphism means avoid write multiple definitions of same methods

What is IS-A relationship and what is HAS-A relationship in java?

A relationship in Java means different relations between two or more classes. For example, if a class Bulb inherits another class Device, then we can say that Bulb is having is-a relationship with Device, which implies Bulb is a device.

In Java, we have two types of relationship:

1. **Is-A relationship:** Whenever one class inherits another class, it is called an IS-A relationship.
2. **Has-A relationship:** Whenever an instance of one class is used in another class, it is called HAS-A relationship.

IS-A Relationship is wholly related to [Inheritance](#). For example – a kiwi is a fruit; a bulb is a device.

- IS-A relationship can simply be achieved by using [extends](#) Keyword.
- IS-A relationship is additionally used for code reusability in Java and to avoid code redundancy.
- IS-A relationship is unidirectional, which means we can say that a bulb is a device, but vice versa; a device is a bulb is not possible since all the devices are not bulbs.
- IS-A relationship is tightly coupled, which means changing one entity will affect another entity.

Advantage of IS-A relationship

- Code Reusability.
- Reduce redundancy.

Example:

```
// Java program to demonstrate the
// working of the Is-A relationship
import java.io.*;
// parent class
class Device {
    private String deviceName;
    public void setDeviceName(String deviceName) {
        this.deviceName = deviceName;
    }

    public String getDeviceName() {
        return this.deviceName + " is a Device";
    }
}
// child class
```

```

class Bulb extends Device {
    public static void main(String gg[]) {
        // parent class can store the reference
        // of instance of child classes
        Device device = new Bulb();

        // set the device name to bulb
        System.out.println("Device name is Bulb");
        device.setDeviceName("Bulb");

        // print the device name
        System.out.println(device.getDeviceName());
    }
}

```

OUTPUT:

Device name is Bulb

Bulb is a Device

[Has-a](#) is a special form of Association where:

- It represents the Has-A relationship.
- It is a unidirectional association i.e. a one-way relationship. For example, here above as shown pulsar motorcycle has an engine but vice-versa is not possible and thus unidirectional in nature.
- In Aggregation, both the entries can survive individually which means ending one entity will not affect the other entity.

// Java Program to Illustrate has-a relation

// Class1

// Parent class

```
public class Car {
```

```
    // Instance members of class Car
```

```
    private String color;
```

```
    private int maxSpeed;
```

```
    // Main driver method
```

```
    public static void main(String[] args)
```

```
{
```

```

        // Creating an object of Car class
        Car nano = new Car();

        // Assigning car object color
        nano.setColor("RED");

        // Assigning car object speed
        nano.setMaxSpeed(329);

        // Calling carInfo() over object of Car class
        nano.carInfo();

        // Creating an object of Maserati class
        Maserati quattroporte = new Maserati();

        // Calling MaseratiStartDemo() over
        // object of Maserati class
        quattroporte.MaseratiStartDemo();
    }

    // Methods implementation

    // Method 1
    // To set the maximum speed of car
    public void setMaxSpeed(int maxSpeed)
    {
        // This keyword refers to current object itself
        this.maxSpeed = maxSpeed;
    }

```

```

// Method 2
// To set the color of car
public void setColor(String color)
{
    // This keyword refers to current object
    this.color = color;
}

// Method 3
// To display car information
public void carInfo()
{
    // Print the car information - color and speed
    System.out.println("Car Color= " + color
        + " Max Speed= " + maxSpeed);
}
}

// Class2
// Child class
// Helper class
class Maserati extends Car {

    // Method in which it is shown
    // what happened with the engine of Puslar
    public void MaseratiStartDemo()
    {
        // Creating an object of Engine type
    }
}

```



```

        // using stop() method
        // Here, MaseratiEngine is name of an object
        Engine MaseratiEngine = new Engine();
        MaseratiEngine.start();
        MaseratiEngine.stop();
    }
}

```

```

// Class 3
// Helper class
class Engine {

```

```

    // Method 1
    // To start a engine
    public void start()
    {
        // Print statement when engine starts
        System.out.println("Started:");
    }

```

```

    // Method 2
    // To stop a engine
    public void stop()
    {
        // Print statement when engine stops
        System.out.println("Stopped:");
    }
}

```

OUTPUT:

Car Color= RED MaxSpeed = 150

Started:

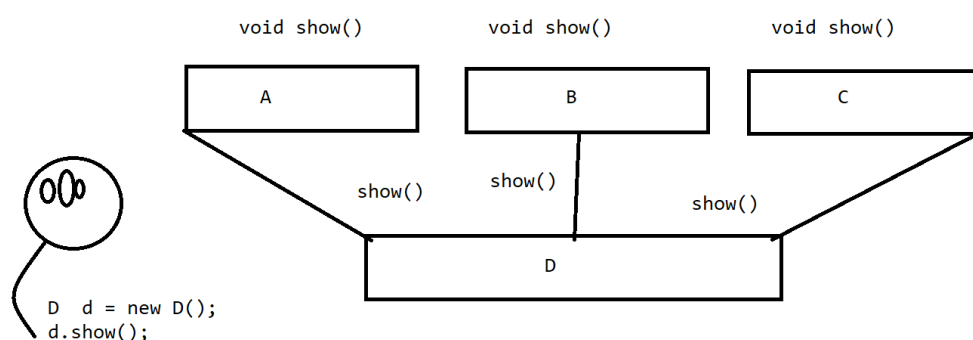
Stopped:

What are multiple inheritances and why java not supports to multiple inheritances using classes?

Because of diamond problem

What is diamond problem in inheritance and how java solves it?

diamond problem means in multiple inheritance there is more than one parent class and single child class so there is possibility in different parent class may be contain same name method and if we create object of child class and try to call the method whose name same in different parent then compiler will get confused and generate compile time error called as diamond problem.



if we think about above diagram we have diamond problem because we have three classes name as A,B and C and we have one child class name as D and all parent contain show() method which is duplicated in class D when we create object of class D and try to call show() method then compiler will generate compile time error

If we want to work with interface we have some important rules or points

if we want to use interface we have interface keyword shown in following code.

Syntax: interface interfacename

```
{  
}
```

How to perform inheritance in java?

```
package org.techhub;
```

```
public class Circle {  
    protected float radius;  
    public void setRadius(float radius)  
    { this.radius=radius;  
    }  
}
```

```
package org.techhub;
```

```
public class Area extends Circle  
{    public float getArea() {  
        return 3.14f*radius*radius;  
    }  
}
```

```
}
```

```
package org.techhub;
```

```
public class Cirm extends Circle {  
    public float getCirm() {  
        return 3.14f*2*radius;  
    }  
}
```

```
}
```

```
package org.techhub;
```

```
public class InheritanceApp {  
    public static void main(String[] args) {  
        Area a = new Area();  
        a.setRadius(3.0f);  
        float result=a.getArea();  
        System.out.println("Area of circle is "+result);  
        Cirm cm = new Cirm();  
        cm.setRadius(4.0f);  
        result = cm.getCirm();  
        System.out.println("Cirm of Circle is "+result);  
    }  
}
```

```
}
```

what happen if parent class contain default constructor and child contain constructor and if we create object of child class?

In java, the default constructor of a parent class called automatically by the constructor of its child class. That means when we create an object of the child class, the parent class constructor executed, followed by the child class constructor executed.

```
// Java Program to Illustrate  
// Invocation of Constructor  
// Calling Without Usage of  
// super Keyword
```

```
// Class 1  
// Super class  
class Base {
```

```
    // Constructor of super class  
    Base()  
    {  
        // Print statement  
        System.out.println(  
            "Base Class Constructor Called ");  
    }  
}
```

```
// Class 2  
// Sub class  
class Derived extends Base {
```

```
    // Constructor of sub class  
    Derived(int a)  
    {  
        // Print statement  
        System.out.println(  
            "Derived Class Constructor Called "+a);  
    }  
}
```

```
// Class 3  
// Main class  
class GFG {  
    // Main driver method
```

```

    public static void main(String[] args){
        // Creating an object of sub class
        // inside main() method
        Derived d = new Derived(3);

        // Note: Here first super class constructor will be
        // called there after derived(sub class) constructor
        // will be called
    }
}

```

OUTPUT:

Base Class Constructor Called

Derived Class Constructor Called 3

What happen if we parent class contain parameterized constructor and if we create object of child class?

it will execute, first parent constructor then child constructor.

// Java Program to Illustrate

// Invocation of Constructor

// Calling Without Usage of

// super Keyword

// Class 1

// Super class

class Base {

// Constructor of super class

Base(int b)

{

// Print statement

System.out.println(

"Base Class Constructor Called "+b);

}

}

// Class 2

// Sub class

class Derived extends Base {

// Constructor of sub class

```

    Derived(int a)
    {
        super(1);
        // Print statement
        System.out.println(
            "Derived Class Constructor Called "+a);
    }
}

// Class 3
// Main class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Creating an object of sub class
        // inside main() method
        Derived d = new Derived(3);

        // Note: Here first super class constructor will be
        // called there after derived(sub class) constructor
        // will be called
    }
}

```

OUTPUT:

```

Base Class Constructor Called 1
Derived Class Constructor Called 3

```

How to pass parameter to parent constructor using a child class constructor?

using super() constructor i.e. need to declare first line of child constructor.

What is final keyword and explain final variable, final method and final class in java?

final keyword

final is non access specifier in java which we can use with variable, with function and with class

i.e. final variable , final method and final class

Final variable

final variable means those variable cannot modify its value once we assign it means final variable work as constant in java means if we want to fix some value then we can use final.

```
public class FinalVarApp
{   public static void main(String x[])
    {
        final int a=100;
        ++a;
        System.out.printf("A is %d\n",a);
    }
}
```

```
C:\Program Files\Java\jdk1.8.0_291\bin>javac FinalVarApp.java
FinalVarApp.java:5: error: cannot assign a value to final variable a
        ++a;
        ^
1 error
```

if we think about above screen short we get compile time error because we have final variable a and we initialize value in it i.e. 100 and we have statement ++a means we try to modify the value of final variable a and it is not possible because we cannot modify value of final variable and we try it so compiler will generate error to us.

final method

final method means method cannot override in child class

Before final method we need to know what is method overriding.

If we want to avoid method overriding in java we have two ways

1. Declare method as final: because when we declare parent class method as final then we cannot override means if we try to override parent class final method in child class then compiler will generate compile time error to us shown in following code means we can say using final method we can avoid overriding.

```
package org.techhub;
class A{
    final void show() {
        System.out.println("I am parent method show ");
    }
}
class B extends A{
    void show() {
        System.out.println("I am child method show ");
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
```

```

        A b1 = new B();
        b1.show();
    }
}

```

if we think about above code it will generate compile time error to us because we have final method show() in parent class and we try to override this method in child class B and it is not possible so compiler will generate compile time error to us.

2. Declare class final : when we declare class as final then we cannot inherit it in any another class means we can say final class has no child class means if final restrict to inheritance means final restrict to overriding because overriding cannot work without inheritance as well as final class is used for when user want to create immutable objects means immutable means we cannot modify its once we assign it

```

package org.techhub;
final class A{
    void show() {
        System.out.println("I am parent method show ");
    }
}
class B extends A{
    void show() {
        System.out.println("I am child method show ");
    }
}
public class OverridingApplication {
    public static void main(String[] args) {
        A b1 = new B();
        b1.show();
    }
}

```

above code generate compile time error to us because we have final class A and we try to inherit it in class B and final class cannot inherit in any another class so we get compile time error

If abstract class cannot create its object then what is purpose of constructor in abstract class?

In Java, even though an abstract class cannot be instantiated, it can have a constructor. The purpose of a constructor in an abstract class is to initialize the object's state and to allow for the chaining of constructors when a subclass is created. When a class doesn't declare any constructor, the compiler creates a default constructor for it, and this is also true for abstract classes.

The main purpose of the abstract class is to create a base class from which many other classes can be derived. An abstract class can have abstract and non-abstract methods, and it can be used as a subclass by another class. Although an abstract class cannot be initiated on its own, it can be used as a superclass for other classes, and the constructor in the abstract class plays a role in this inheritance hierarchy.

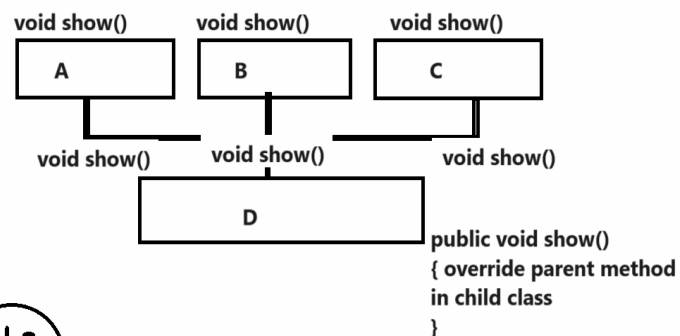
How we can achieve multiple inheritances using inheritance explain with example?

We can not achieve multiple inheritance only using inheritance we need to implement interface to achieve multiple inheritance in java.

```

1 package org.techhub;
2 interface A{
3     void show();
4 }
5 interface B{
6     void show();
7 }
8 class C{
9     void show() {
10         System.out.println("I am C method");
11     }
12 }
13 class D extends C implements A,B{
14     public void show() {
15         System.out.println("I am D method");
16     }
17 }
18 public class InterfaceTestApp {
19     public static void main(String[] args) {
20         D d1 = new D();
21         d1.show();
22     }
23 }
24

```



What is diff between abstract class and interface?

Interface	Abstract class
Interface use 100% abstraction	Abstract class use partial abstraction
Interface every method is by default public abstract	Need to declare manually abstract method in abstract class using abstract keyword
Interface variables are by default public static final	Abstract class variable not by default public static final
Interface is used for achieve multiple inheritance	Using abstract class we cannot achieve multiple inheritance
Interface need to implements	Abstract class need to extends or inherit
Interface cannot have constructor	Abstract class can have constructor
Interface method cannot declare as protected	Abstract class abstract method can declare as protected

Use interface keyword for interface declaration	Use class keyword for abstract class declaration
Interface contain only abstract methods ,static method and default	Abstract class can use all types of methods

Why interface variables are final?

Because we use variable within interface for constant purpose and interface cannot create its object so if we want to access variable or function without object then it must be static so this major reason interface is variable and final.

static because we can use it by using interface name, final because we cannot modify its value, and public because it can use within package as well as outside of package.

```

1 package org.techhub;
2
3 interface Area{
4     float PI=3.14f;//public static final float PI
5 }
6 public class InterfaceApplication {
7
8     public static void main(String[] args) {
9         System.out.println(Area.PI);
10    }
11 }

```

If we think about above screen short we access Area.PI because PI is static variable so we can access it using interface name.

Can we declare interface method as final?

No, we cannot declare interface method as final because interface method is internally public abstract and abstract method must be override and final method cannot be override so this is major reason we cannot declare interface method as final and if we declare it as final compiler generate compile time error to us

```

3 interface Area{
4     float PI=3.14f;//public static final float PI
5     final float getArea();
6 }

```

above code generate compile time error to us because we have method name as getArea() within interface and we mark as final so it is not allow

Can we declare interface method as private, protected or static explain with reasons?

No, we cannot declare interface method as private

There are two reasons

1. private access specifier not support to method overriding as well as inheritance and interface method must be override so we cannot declare interface method as private
2. If we declare interface method then it is by default public abstract and when we declare it as private then compiler consider it as private public abstract and in java there is only private access specifier or public access specifier but there is not private public access specifier and if we declare as private so compiler will generate compile time error to us show in following code.

```
interface ABC{
    void show();//public abstract void show();
}

interface ABC
{
    private void show();//private public abstract void show();
}
```



compile generate error here

if we think about above screen short we have first code is

```
interface ABC{
    void show();//public abstract void show()
}
```

here internally show() method consider as public abstract by default and we have one more code

```
interface ABC{
    private void show();//private public abstract void show()
}
```

if we think above code here we use private access specifier with interface method so internally interface method use public access specifier and private cannot override on public so compiler consider user define method with private public access specifier at time so java suggest you can use either private or public but not both at same time so this is major we cannot declare interface method as private

Q. Can we declare interface method as protected?

No we cannot declare interface method as protected and if we declare it as protected then compiler will generate compile time error.

```
interface Area{
    protected void show();
}
```

Q. Why we cannot declare interface method as protected even it is abstract because we can declare abstract method as protected in abstract class?

when we declare abstract method in abstract class then abstract method us default access specifier by default and when we declare it as protected then protected get override on default so compiler consider only once access with abstract method in abstract class so not generate compile time error.

Example:

```
abstract class A{           //internal meaning
    abstract void show(); // default abstract void show();
}
```

when we use protected with abstract method in abstract class then it look like as

```
abstract class A{           //internal meaning
    protected abstract void show(); // protected abstract void show();
}
```

Here protected keyword get override on default so compiler not generate error because compiler found only one access specifier with show () method

When we declare interface method as protected then

```
interface A{ //internal meaning
    void show(); //public abstract void show();
}
```

when we declare it as protected

```
interface A{
    protected void show(); //protected public abstract void show();
}
```

here compiler consider it as protected public so compiler found two access specifier with method at time and as per rule we cannot use two access specifier at time with function so compiler will generate compile time error to us.

Can we extend interface to interface explain with example?

Yes we can perform inheritance between interface to interface and for that we have extends keyword

```

1 package org.techhub;
2 interface A{
3     void show();
4 }
5 interface B extends A{
6     void display(); //interface B contain two methods i.e show() and display()
7     //because B is child of A
8 }
9 class C implements B{
10     public void show() {
11         System.out.println("I am A method");
12     }
13     public void display() {
14         System.out.println("I am B method");
15     }
16 }
17 public class InterfaceTestApp {
18     public static void main(String[] args) {
19         C c1 = new C();
20         c1.show();
21         c1.display();
22     }
23 }

```

Output:

Screenshot of InterfaceTestApp.java Application

```

I am A method
I am B method

```

can we use class within interface?

Yes, in Java, it is possible to define a class inside an interface. This can be useful when the methods of the interface use this class and it is not used anywhere else. Here's an example:

This allows for the encapsulation of the class within the interface, providing a way to organize related code. However, it's important to note that you cannot access a nested class directly; you need to access (implement) the nested class using the inner class or by using the name of the class holding this nested class.

Yes, definitely you can declare a class inside an interface.
see the below example:-

Interface I

```

{
class A
{
void m1(){
System.out.println("inside m1");
}
void m2(){
System.out.println("inside m2");
}
}

```

```
}  
}  
}
```

```
Class Test{
```

```
public static void main(String args[])  
I.A ia = new I.A();  
ia.m1();  
ia.m2();  
}
```

In this case, the inner class is becoming as static inner class inside interface where we can access the members of inner class like we do in case of static inner class.

Can we declare interface as static?

Static Methods in **Interface** are those methods, which are defined in the interface with the keyword static. Unlike other methods in Interface, these static methods contain the complete definition of the function and since the definition is complete and the method is static, therefore these methods cannot be overridden or changed in the implementation class.

Similar to [Default Method in Interface](#), the static method in an interface can be defined in the interface, but cannot be overridden in Implementation Classes. To use a static method, Interface name should be instantiated with it, as it is a part of the Interface only.

Below programs illustrate static methods in interfaces:

Program 1: To demonstrate use of Static method in Interface.

In this program, a simple static method is defined and declared in an interface which is being called in the main() method of the Implementation Class InterfaceDemo.

Unlike the default method, the static method defines in Interface hello(), cannot be overridden in implementing the class.

```
// Java program to demonstrate  
// static method in Interface.
```

```
interface NewInterface {
```

```
    // static method  
    static void hello()  
    {
```

```
        System.out.println("Hello, New Static Method Here");
```

```

    }

    // Public and abstract method of Interface
    void overrideMethod(String str);
}

// Implementation Class
public class InterfaceDemo implements NewInterface {

    public static void main(String[] args)
    {
        InterfaceDemo interfaceDemo = new InterfaceDemo();

        // Calling the static method of interface
        NewInterface.hello();

        // Calling the abstract method of interface
        interfaceDemo.overrideMethod("Hello, Override Method here");
    }

    // Implementing interface method

    @Override
    public void overrideMethod(String str)
    {
        System.out.println(str);
    }
}

```

OUTPUT:

Hello, New Static Method Here

Hello, Override Method here

POLYMORPHISM

what is polymorphism and explain types of polymorphism?

Polymorphism means if same thing perform different behaviour in different situation as per his requirement called as polymorphism.

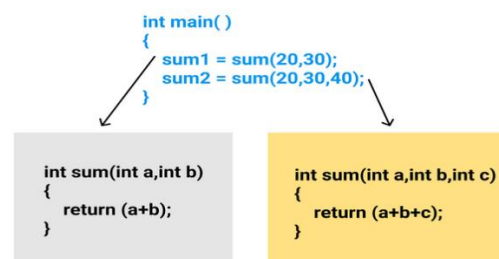
Example: You can say mobile phone is best example of polymorphism means you can use it as phone, as calculator as camera, as tv, as audio device etc means we can use single device for multiple purpose called as polymorphism.

In simple words, we can define polymorphism as the ability of a message to be displayed in more than one form.

A person at the same time can have different characteristics. A man at the same time is a father, a husband, and an employee. So the same person possesses different behavior in different situations. This is called polymorphism. An operation may exhibit different behaviors in different instances. The behavior depends upon the types of data used in the operation. JAVA supports function overloading.

- **Function Overloading:** Function overloading is using a single function name to perform different types of tasks. Polymorphism is extensively used in implementing inheritance.

Example: Suppose we have to write a function to add some integers, sometimes there are 2 integers, and sometimes there are 3 integers. We can write the Addition Method with the same name having different parameters, the concerned method will be called according to parameters.



polymorphism means if same thing performs different behaviour in different situation as per his requirement called as polymorphism.

Polymorphism means if we use the same thing for different forms or different purpose called as polymorphism.

There are two types of polymorphism:

- A) Compile time polymorphism: means if object bounded with functionality at program compile time called as compile time polymorphism.

There are two ways to achieve compile time polymorphism in c++ :

- Using function overloading.
- B) Run time polymorphism: means if object bounded with functionality at program run time called as run time polymorphism.
- Implemented by using function overriding technique

1 Compile time polymorphism:

compile time polymorphism means if object bounded with functionality at program compile time called as compile time polymorphism. It means we decide function calling at compile time.

Compile time implementation or compile time polymorphism:

- Using function overloading:

Function overloading means if we have same name function with a different data type with a different parameter list with a different parameter sequence or different number of parameter called as function overloading.

Function call depends on the parameter passed to function. Return type not considered in function overloading means you can give different return type with every function definition.

```
// Java Program for Method overloading
```

```
// By using Different Types of Arguments
```

```
// Class 1
```

```
// Helper class
```

```
class Helper {
```

```
    // Method with 2 integer parameters
```

```
    static int Multiply(int a, int b)
```

```
    {
```

```
        // Returns product of integer numbers
```

```
        return a * b;
```

```
    }
```

```
    // Method 2
```

```
    // With same name but with 2 double parameters
```

```
    static double Multiply(double a, double b)
```

```
    {
```

```
        // Returns product of double numbers
```

```
        return a * b;
```

```
    }
```

```
}
```

```
// Class 2
```

```
// Main class
```

```

class GFG {
    // Main driver method
    public static void main(String[] args)
    {
        // Calling method by passing
        // input as in arguments
        System.out.println(Helper.Multiply(2, 4));
        System.out.println(Helper.Multiply(5.5, 6.3));
    }
}

```

OUTPUT:

8

34.65

Runtime Polymorphism in Java

It is also known as Dynamic Method Dispatch. It is a process in which a function call to the overridden method is resolved at Runtime. This type of polymorphism is achieved by Method Overriding. Method overriding, on the other hand, occurs when a derived class has a definition for one of the member functions of the base class. That base function is said to be **overridden**.

// Java Program for Method Overriding

```

// Class 1
// Helper class
class Parent {

    // Method of parent class
    void Print()
    {

        // Print statement
        System.out.println("parent class");
    }
}

```

```
// Class 2
// Helper class
class subclass1 extends Parent {

    // Method
    void Print() { System.out.println("subclass1"); }
}
```

```
// Class 3
// Helper class
class subclass2 extends Parent {
    // Method
    void Print(){
        // Print statement
        System.out.println("subclass2");
    }
}
```

```
// Class 4
// Main class
class GFG {

    // Main driver method
    public static void main(String[] args)
    {

        // Creating object of class 1
        Parent a;

        // Now we will be calling print methods
        // inside main() method

        a = new subclass1();
        a.Print();

        a = new subclass2();
        a.Print();
    }
}
```

OUTPUT:
subclass1
subclass2

Is it true super () constructor can call immediate parent class?
super() constructor call immediate parent class constructor.

Is it true by default java use single level inheritance?

yes, in java default inheritance is single level inheritance. This means that a class can inherit properties from only one class. when class inherits another class, it is known as single inheritance.

```
class Animal {  
    // Animal class properties and methods  
}  
class Dog extends Animal {  
    // Dog class inherits from Animal class  
}
```

Is it true we can perform overloading in inheritance?

Yes, it is true that we can perform overloading in inheritance in Java. Method overloading in Java allows a class to have more than one method having the same name if their parameter lists are different. This concept can be applied to inherited methods as well.

When a subclass inherits a method from a superclass, it can also overload that method by providing a different implementation with a different parameter list. This allows the subclass to have multiple methods with the same name but different parameters, providing more flexibility and customization.

```
class Parent {  
    void display(int a) {  
        // Method implementation  
    }  
}  
  
class Child extends Parent {
```

```
void display(int a, int b) {  
    // Overloaded method implementation  
}  
}
```

In this example, the `Child` class overloads the `display` method inherited from the `Parent` class by providing a different parameter list.